UCS503 – Software Engineering

Driver Drowsiness Detection and Alert System

Prototype-Stage Project Report

Submitted by:

1024030725 Arshita Gupta
1024030730 Tarushi Goyal
1024030746 Chahana Goyal

Group: 3C53 BE Third Year, COE

Submitted to:

Dr. Paramveer Sidhu

Mid-Semester Evaluation – Prototype Stage

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Project Objectives . . . 3

2 Methodology and Design **5**
- 2.1 System Architecture . . . 5
  - 2.1.1 High-Level Design . . . 5
  - 2.1.2 Technical Stack . . . 6
- 2.2 Implementation Details . . . 6
  - 2.2.1 Home Screen and Navigation . . . 6
  - 2.2.2 Camera Monitoring Screen . . . 6
  - 2.2.3 Face Landmark Detection Helper . . . 7
  - 2.2.4 EAR and MAR Processing . . . 7
  - 2.2.5 Prototype Workflow . . . 7

3 Results and Evaluation **9**
- 3.1 Testing Methodology . . . 9
- 3.2 Current Prototype Status . . . 9
- 3.3 Performance Metrics . . . 10
- 3.4 Analysis . . . 11

4 Conclusion **12**
- 4.1 Summary of Work . . . 12
- 4.2 Key Findings . . . 12
- 4.3 Future Work and Recommendations . . . 12
- 4.4 Final Remarks . . . 13

Chapter 1

Introduction

1.1 Background and Context

Driver fatigue can reduce attention and reaction time and may increase the risk of road accidents. This project develops a camera-based Android application intended to identify signs associated with driver drowsiness. It uses facial landmarks to calculate the Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR), then evaluates eye closure and mouth opening over time.

The current prototype is a native Android application developed in Kotlin using Jetpack Compose, CameraX, and MediaPipe Face Landmarker. The dashboard provides live monitoring, displays the current state and measured EAR/MAR values, and shows a summary screen after monitoring is stopped. The project proposal also identifies timely alerts and measurable evaluation as goals; an audible or vibration alert is not yet generated by the current prototype.

1.1.1 Problem Statement

Early signs of fatigue, such as prolonged eye closure and repeated yawning, may go unnoticed by a driver. A camera-based Android application can investigate these indicators using commonly available mobile hardware.

The application now calculates EAR and MAR from facial landmarks and uses duration-based thresholds to classify the current state. The dashboard displays states such as Awake, Eyes closed, DROWSY!, Mouth open, and YAWNING!. A session summary is shown after monitoring stops and includes duration, event counts, longest observed eye closure, recorded EAR/MAR readings, and alert-event history. The application does not currently generate an audible or vibration alert, so the alert mechanism described in the proposal remains incomplete.

1.2 Project Objectives

The project objectives are:

1. Develop an Android application with a live monitoring dashboard and a session summary screen.

2. Access the front-facing camera using CameraX and pass image frames to an analysis pipeline.

3. Integrate MediaPipe Face Landmarker to obtain facial landmarks.
4. Calculate Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) from selected facial landmarks.
5. Apply duration-based logic to classify prolonged eye closure and mouth opening.
6. Record session readings and event counts locally and display a summary after monitoring.
7. Complete the audio or vibration alert mechanism and evaluate the application through documented tests.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

The workflow begins when the user selects Start Monitoring and grants camera permission. CameraX provides frames from the front-facing camera; the CameraAnalyzer converts each frame to a bitmap, rotates and mirrors it, and passes it to the MediaPipe Face Landmarker helper. The returned landmarks are passed to DrowsinessAnalyzer, which calculates EAR and MAR and evaluates how long the eyes or mouth remain beyond the configured thresholds.

The dashboard displays the current classification and live measurements. When monitoring is stopped, the application displays a session summary and saves session data locally as JSON. The dashboard also provides access to past sessions. Audio or vibration alert generation is not currently implemented, and the alert-event display should not be interpreted as proof that a sound or vibration was triggered.

The overall architecture of the proposed system is shown in Figure 2.1.

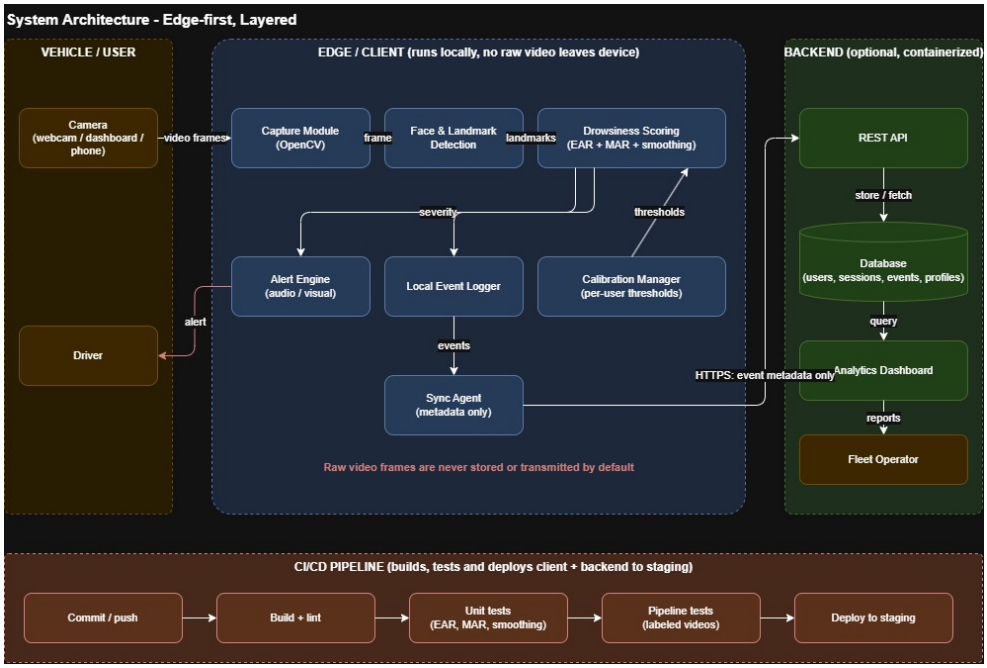


Figure 2.1: System Architecture of the Driver Drowsiness Detection System

2.1.2 Technical Stack

Table 2.1: Technologies used in the current prototype

Technology	Role
Kotlin	Main programming language for the Android application.
Android Studio	Development environment.
Jetpack Compose	User-interface layout and screen components.
CameraX	Front-camera preview and image analysis.
MediaPipe Face Landmarker	Facial landmark detection helper.
Gradle	Build configuration and dependency management.

Session data is stored locally on the device in JSON files using Android's private application storage and AtomicFile. A remote database or backend is not part of the current implementation.

2.2 Implementation Details

2.2.1 Home Screen and Navigation

The `MainActivity.kt` hosts the Jetpack Compose interface and controls monitoring-session state. The dashboard includes Start Monitoring and Stop Monitoring actions, navigation to past sessions, and a summary view after a session ends. The current interface includes the dashboard and history/summary workflow; a separate fully implemented Statistics screen and Settings screen are not evidenced in the shared files.

2.2.2 Camera Monitoring Screen

The `CameraAnalyzer.kt` processes front-camera frames through CameraX using the `STRATEGY_KEEP_ONLY_LATEST` back-pressure strategy. Each frame is converted to a bitmap, rotated and mirrored, passed to MediaPipe, and closed after processing. The dashboard displays the live camera preview, current status, session duration, EAR, MAR, eye-closure duration, event counts, and recent alert-event entries while monitoring is running.

2.2.3 Face Landmark Detection Helper

The `FaceLandmarkerHelper.kt` initializes MediaPipe Face Landmarker in live-stream mode, configures detection for one face, and sets confidence thresholds. It submits frames asynchronously using `detectAsync` and passes results or errors through callbacks. The helper also provides a close method for cleanup.

The helper supplies facial landmarks to the analysis callback. EAR/MAR calculation and state classification are handled by `DrowsinessAnalyzer.kt`, while session duration, event counts, readings, persistence, and summary navigation are handled by `MainActivity.kt` and the session data model. The application does not currently trigger an audible or vibration alert.

2.2.4 EAR and MAR Processing

The current `DrowsinessAnalyzer.kt` calculates an EAR for each eye using selected MediaPipe landmarks and averages the two values. It calculates MAR by comparing the vertical distance between mouth landmarks with the horizontal mouth width. The EAR calculation uses the following geometric form:

$$\text{EAR} = \frac{d(p_2, p_6) + d(p_3, p_5)}{2d(p_1, p_4)}$$

where $d(p_i, p_j)$ is the Euclidean distance between two landmark points. The implementation uses MediaPipe landmark indices for both eyes and the mouth.

The current analyzer uses an EAR threshold of 0.22 and a sustained eye-closure duration of 1.5 seconds to classify the state as DROWSY!. It uses a MAR threshold of 0.35 sustained for 1 second to classify the state as YAWNING!. When the eye ratio is below threshold but the duration has not yet elapsed, the displayed state is Eyes closed; otherwise the state is Awake when neither condition applies. These are prototype thresholds and have not been presented as universally valid or clinically validated settings.

2.2.5 Prototype Workflow

The current workflow includes camera-frame analysis, facial landmark detection, EAR/MAR calculation, duration-based state classification, session readings, event counts, local session storage, and a summary screen. Audio or vibration alert generation remains subsequent work.

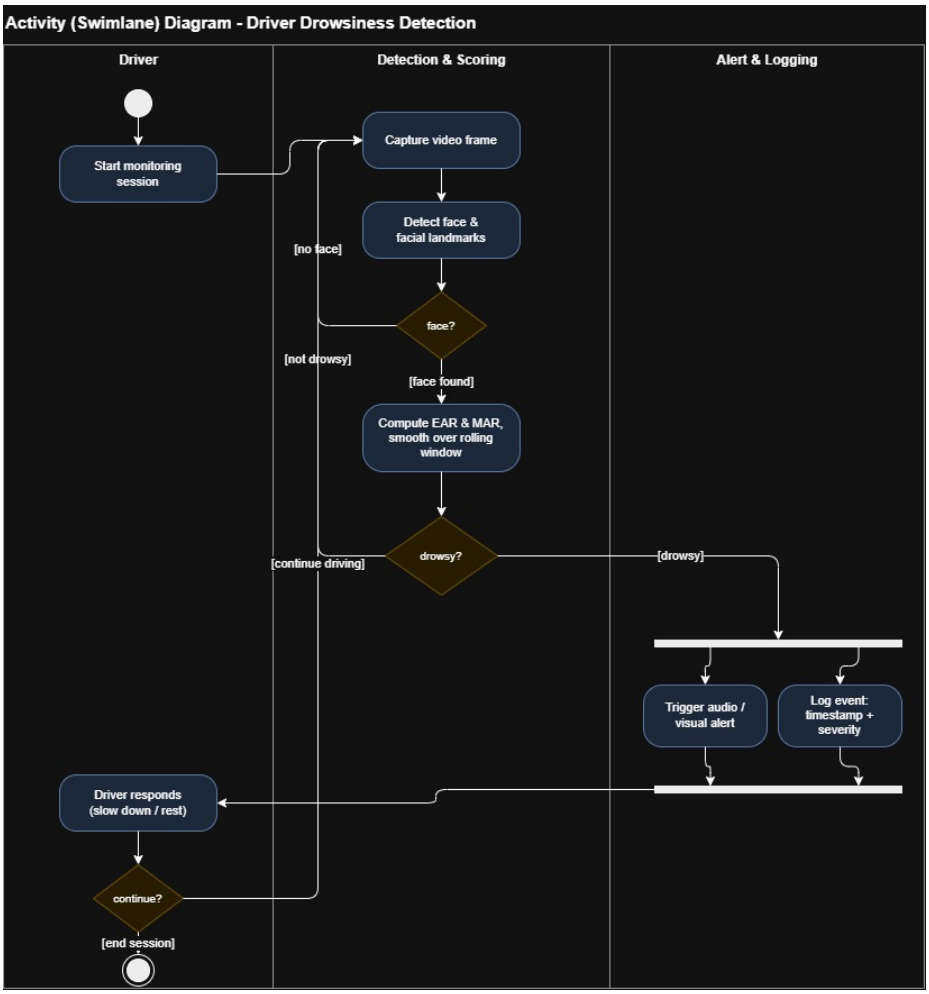


Figure 2.2: Swimlane Diagram of the Monitoring Workflow

Chapter 3

Results and Evaluation

3.1 Testing Methodology

The prototype should be tested in stages:

- **Build testing:** confirm that the Android project compiles successfully.
- **Interface testing:** check the home screen and navigation to the monitoring screen.
- **Camera testing:** verify permission handling, camera preview, frame delivery, and cleanup when monitoring stops.
- **Landmark testing:** verify that MediaPipe initializes on the target device and returns landmarks for a visible face.
- **Algorithm testing:** check EAR/MAR values and state changes for open eyes, prolonged eye closure, and sustained mouth opening.
- **Session testing:** verify that stopping monitoring opens the summary and that session readings and event counts are saved and reloaded.
- **Alert testing (pending):** verify audio or vibration output after an alert mechanism has been implemented.
- **Performance testing:** measure processing speed, classification behaviour, and false-positive/false-negative cases on a documented test set.

3.2 Current Prototype Status

Table 3.1 summarizes the implementation status supported by the current source files and the reported dashboard behaviour.

Table 3.1: Implementation status at the prototype stage

Component	**Status in shared code**
Dashboard and live monitoring controls	Implemented
Camera permission and CameraX frame pipeline	Implemented in source
MediaPipe facial landmark detection	Integrated through helper
EAR and MAR calculations	Implemented in DrowsinessAnalyzer
Duration-based drowsiness/yawning classification	Implemented with prototype thresholds
Live status, ratios, and eye-closure duration	Displayed on dashboard
Session summary and reading graph	Displayed after monitoring stops
Local session persistence	JSON files stored in app-private storage
Audio/vibration alert output	Not implemented
Performance/accuracy evaluation	Not yet documented with measured results

3.3 Performance Metrics

The current source code establishes the configured thresholds and the session-data flow, but it does not establish verified detection accuracy, alert latency, sustained frame rate, or false-alarm rate. These metrics should only be reported after measurements have been collected on a specified device using a repeatable test procedure. No audible or vibration alert is currently generated, so alert latency cannot yet be evaluated.

Table 3.2: Metrics planned for a later test cycle

Metric	**Measurement approach**
Landmark detection success	Proportion of test frames where landmarks are returned for a visible face.
Processing frame rate	Frames processed per second on the selected device.
EAR/MAR behaviour	Record ratios for open eyes, eye closure, and mouth opening.
State classification	Compare displayed states with manually labelled test cases.
Alert latency	Measure event onset to audible/vibration output after that output is implemented.
False-alarm rate	Count incorrect drowsiness/yawning classifications per hour of test footage.

3.4 Analysis

The current implementation has progressed beyond initial camera and landmark integration: EAR and MAR are calculated, the dashboard classifies eye closure and yawning based on configured duration thresholds, and a session summary is displayed after monitoring. Session data includes sampled readings, duration, drowsiness/yawning counts, longest observed eye closure, and event history, and is saved locally.

The main outstanding feature relative to the proposal is an actual audible or vibration alert. The threshold-based classification and event counts also need systematic testing across lighting, camera position, users, and ordinary versus simulated-drowsy conditions. Accuracy, false-alarm rate, and latency should not be claimed until measured.

Chapter 4

Conclusion

4.1 Summary of Work

At the prototype stage, the project has a live Android monitoring dashboard, CameraX-based frame processing, MediaPipe facial landmark detection, EAR/MAR calculations, duration-based state classification, and a post-session summary screen. Session readings and counts are stored locally and can be reviewed through the history/summary workflow.

An audible or vibration alert is not yet generated. The system's current classifications are threshold-based prototype outputs and require systematic evaluation before any claim of reliable real-world drowsiness detection.

4.2 Key Findings

- The dashboard displays live EAR/MAR measurements and changes the status according to eye-closure and mouth-opening duration thresholds.
- A summary screen presents the session duration, event counts, longest eye closure, recorded ratios, and event history after monitoring stops.
- Session information is saved locally in JSON files.
- An actual audio or vibration alert and formal performance evaluation remain incomplete.

4.3 Future Work and Recommendations

The planned next steps are:

1. Implement an audible or vibration alert when the drowsiness condition is detected and verify it on the target device.
2. Test EAR/MAR thresholds and duration logic with controlled examples and different lighting and camera positions.
3. Verify that session readings, event counts, and summaries remain correct across multiple monitoring sessions.
4. Improve and test history/statistics views as required by the project scope.

5. Measure processing speed, detection behaviour, false-alarm rate, and alert latency once alert output is available.

6. Document known limitations and avoid presenting the prototype as a certified driving-safety system.

4.4 Final Remarks

This prototype implements the central monitoring flow: camera capture, facial landmark analysis, EAR/MAR measurements, threshold-based state classification, local session recording, and a post-monitoring summary. The alert output and formal performance evaluation remain outstanding in comparison with the project proposal. The application is an academic prototype and should not be treated as a certified safety system or a substitute for safe driving practices.